

Arrays, Multidimensional Matrices and Memory Layout in C

📌 Definition to Remember

An array is a collection of elements of identical data types stored in contiguous memory locations, offering $O(1)$ random access via index arithmetic.

1. Concept Overview & Fundamentals 🧠

Arrays, Multidimensional Matrices and Memory Layout in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Code is compiled directly into native machine instructions without virtual machine overhead.
- Memory is strictly partitioned into **Code/Text Segment**, **Data Segment (Initialized/BSS)**, **Stack**, and **Heap**.
- Direct pointer and register manipulation provides deterministic performance critical for operating systems, embedded hardware, and database kernels.

2. Technical Architecture & Memory Mechanics ⚙️

Memory Layout & Storage Organization

```
+-----+
|                STACK SEGMENT                |
| - Local variables, function call stack frames, parameters |
| - High memory growing downwards towards Heap |
+-----+
|
| v
| ^
|
+-----+
|                HEAP SEGMENT                |
| - Dynamic memory allocations via malloc(), calloc() |
| - Low memory growing upwards towards Stack |
+-----+
|                INITIALIZED DATA SEGMENT      |
| - Global and static variables with non-zero initial values |
+-----+
|                BSS SEGMENT (UNINITIALIZED)    |
| - Global and static variables initialized to zero |
+-----+
|                CODE / TEXT SEGMENT            |
| - Read-only executable binary instructions |
+-----+
```

Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers dynamic scalability	Stack: O(1) push/pop; Heap: O(1) alloc
Type Checking	Static Typing	Compiler verifies types during compilation	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memory address	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops, and call stacks	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Arrays\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.
- **Embedded & IoT Devices:** Microcontrollers with constrained RAM (8KB-64KB) require C's predictable footprint.
- **Database Engine Kernels:** High-throughput storage engines (PostgreSQL, SQLite, MySQL InnoDB) rely on low-level buffer pool managers written in C.
- **Game Engine Foundations:** Graphics rasterization, physics engines, and GPU interfaces utilize C/C++ memory arrays for zero-overhead computation.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: array max min

Problem Description & Objective:

Implementation of array max min

C Code Implementation:

```
/*
Create a program to find the maximum and minimum
element in an array.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to find the maximum and minimum element in an array.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr[5], i;
    int max, min;

    printf("Enter 5 elements:\n");

    for(i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }

    max = arr[0];
    min = arr[0];

    for(i = 1; i < 5; i++) {

        if(arr[i] > max) {
            max = arr[i];
        }

        if(arr[i] < min) {
            min = arr[i];
        }
    }

    printf("Maximum element = %d\n", max);
    printf("Minimum element = %d\n", min);

    return 0;
}
```

Problem 2: array occurrences

Problem Description & Objective:

Implementation of array occurrences

C Code Implementation:

```
/*
Create a program to find number of occurrences
of an element in an array.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to find number of occurrences of an element in an array.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr[10], i, num, count = 0;

    printf("Enter 10 elements:\n");

    for(i = 0; i < 10; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &num);

    for(i = 0; i < 10; i++) {

        if(arr[i] == num) {
            count++;
        }
    }

    printf("Number of occurrences = %d\n", count);

    return 0;
}
```

Problem 3: array palindrome

Problem Description & Objective:

Implementation of array palindrome

C Code Implementation:

```

/*
Create a program to check if the array is palindrome or not.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to check if the array is palindrome or not.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr[5], i;
    int isPalindrome = 1;

    printf("Enter 5 elements:\n");

    for(i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }

    for(i = 0; i < 5 / 2; i++) {

        if(arr[i] != arr[5 - i - 1]) {
            isPalindrome = 0;
            break;
        }
    }

    if(isPalindrome == 1) {
        printf("Array is Palindrome.\n");
    } else {
        printf("Array is NOT Palindrome.\n");
    }

    return 0;
}

```

Problem 4: array sorted check

Problem Description & Objective:

Implementation of array sorted check

C Code Implementation:

```

/*
Create a program to check if the given array is sorted.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to check if the given array is sorted.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr[5], i;
    int isSorted = 1;

    printf("Enter 5 elements:\n");

    for(i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }

    for(i = 0; i < 4; i++) {

        if(arr[i] > arr[i + 1]) {
            isSorted = 0;
            break;
        }
    }

    if(isSorted == 1) {
        printf("Array is Sorted.\n");
    } else {
        printf("Array is NOT Sorted.\n");
    }

    return 0;
}

```

Problem 5: array sum average

Problem Description & Objective:

Implementation of array sum average

C Code Implementation:

```

/*
Create a program to find the sum and average
of all elements in an array.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to find the sum and average of all elements in an array.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr[5], i;
    int sum = 0;
    float average;

    printf("Enter 5 elements:\n");

    for(i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
        sum = sum + arr[i];
    }

    average = sum / 5.0;

    printf("Sum = %d\n", sum);
    printf("Average = %.2f\n", average);

    return 0;
}

```

Problem 6: copy char array pointer

Problem Description & Objective:

Implementation of copy char array pointer

C Code Implementation:

```

/*
Write a function that uses pointer arithmetic
to copy an array of char into another.
*/

#include <stdio.h>

// Function Definition
void copyString(const char *source, char *destination) {
    /*
    * Logic:
    * 1. Uses pointer arithmetic to traverse the source string.
    * 2. Continues copying characters until the source pointer dereferences to the null terminator
    ('\0').
    * 3. Inside the loop, copies the character at *source to *destination, then increments both
    pointers.
    * 4. Once complete, writes the null terminator ('\0') to the end of the destination buffer.
    */
    while(*source != '\0') {
        *destination = *source;

        source++;
        destination++;
    }

    *destination = '\0';
}

int main() {
    char str1[100], str2[100];

    printf("Enter a string: ");
    scanf("%99s", str1);

    copyString(str1, str2);

    printf("Copied string = %s\n", str2);

    return 0;
}

```

Problem 7: delete array element

Problem Description & Objective:

Implementation of delete array element

C Code Implementation:

```

/*
Create a program to return a new array deleting
a specific element.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * 1. Allocates static buffers of size 10 for both the original (arr) and the filtered (newArr)
arrays.
    * 2. Prompts the user for the number of terms 'n' and validates that 1 <= n <= 10 to prevent array
out-of-bounds writes.
    * 3. Confirms that scanf reads succeed successfully to ensure no uninitialized variables are
processed.
    * 4. Prompts the user for the 'element' to be deleted.
    * 5. Loops through 'arr' and copies any values that do NOT equal 'element' into 'newArr' at index
'j'.
    * 6. Prints the resulting 'newArr' array of size 'j'.
    */
    int arr[10], newArr[10];
    int i, n, element, j = 0;

    printf("Enter number of elements (1-10): ");
    if (scanf("%d", &n) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    if (n < 1 || n > 10) {
        printf("Invalid number of elements. Must be between 1 and 10.\n");
        return 1;
    }

    printf("Enter array elements:\n");

    for(i = 0; i < n; i++) {
        if (scanf("%d", &arr[i]) != 1) {
            printf("Invalid element input.\n");
            return 1;
        }
    }

    printf("Enter element to delete: ");
    if (scanf("%d", &element) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    for(i = 0; i < n; i++) {

        if(arr[i] != element) {
            newArr[j] = arr[i];
            j++;
        }
    }
}

```

```
printf("New array:\n");

for(i = 0; i < j; i++) {
    printf("%d ", newArr[i]);
}

return 0;
}
```

Problem 8: diagonal sum

Problem Description & Objective:

Implementation of diagonal sum

C Code Implementation:

```

/*
Create a program to find the sum of two diagonal elements.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to find the sum of two diagonal elements.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr[3][3];
    int i, j;
    int primarySum = 0;
    int secondarySum = 0;

    printf("Enter 3x3 matrix elements:\n");

    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            scanf("%d", &arr[i][j]);
        }
    }

    // Primary diagonal sum
    for(i = 0; i < 3; i++) {
        primarySum = primarySum + arr[i][i];
    }

    // Secondary diagonal sum
    for(i = 0; i < 3; i++) {
        secondarySum = secondarySum + arr[i][2 - i];
    }

    printf("Primary Diagonal Sum = %d\n", primarySum);
    printf("Secondary Diagonal Sum = %d\n", secondarySum);

    return 0;
}

```

Problem 9: merge sorted arrays

Problem Description & Objective:

Implementation of merge sorted arrays

C Code Implementation:

```

/*
Create a program to merge two sorted arrays.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to merge two sorted arrays.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr1[5], arr2[5], merged[10];
    int i, j, k = 0;

    printf("Enter 5 sorted elements for first array:\n");

    for(i = 0; i < 5; i++) {
        scanf("%d", &arr1[i]);
    }

    printf("Enter 5 sorted elements for second array:\n");

    for(i = 0; i < 5; i++) {
        scanf("%d", &arr2[i]);
    }

    i = 0;
    j = 0;

    while(i < 5 && j < 5) {

        if(arr1[i] < arr2[j]) {
            merged[k] = arr1[i];
            i++;
        } else {
            merged[k] = arr2[j];
            j++;
        }

        k++;
    }

    while(i < 5) {
        merged[k] = arr1[i];
        i++;
        k++;
    }

    while(j < 5) {
        merged[k] = arr2[j];
        j++;
        k++;
    }
}

```

```
printf("Merged array:\n");

for(i = 0; i < 10; i++) {
    printf("%d ", merged[i]);
}

return 0;
}
```

Problem 10: reverse array

Problem Description & Objective:

Implementation of reverse array

C Code Implementation:

```
/*
Create a program to reverse an array.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to reverse an array.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr[5], i;

    printf("Enter 5 elements:\n");

    for(i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Reversed array:\n");

    for(i = 4; i >= 0; i--) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Problem 11: search 2d array

Problem Description & Objective:

Implementation of search 2d array

C Code Implementation:

```
/*
Create a program to search an element in a 2-D array.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to search an element in a 2-D array.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr[3][3];
    int i, j, num, found = 0;

    printf("Enter 3x3 matrix elements:\n");

    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            scanf("%d", &arr[i][j]);
        }
    }

    printf("Enter element to search: ");
    scanf("%d", &num);

    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            if(arr[i][j] == num) {
                printf("Element found at position [%d][%d]\n", i, j);
                found = 1;
            }
        }
    }

    if(found == 0) {
        printf("Element not found.\n");
    }

    return 0;
}
```

Problem 12: sum average 2d array

Problem Description & Objective:

Implementation of sum average 2d array

C Code Implementation:

```

/*
Create a program to do sum and average
of all elements in a 2-array.
*/

#include <stdio.h>

int main() {
    /*
    * Logic:
    * - Goal: Create a program to do sum and average of all elements in a 2-array.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    int arr[3][3];
    int i, j, sum = 0;
    float average;

    printf("Enter 3x3 matrix elements:\n");

    for(i = 0; i < 3; i++) {
        for(j = 0; j < 3; j++) {
            scanf("%d", &arr[i][j]);

            sum = sum + arr[i][j];
        }
    }

    average = sum / 9.0;

    printf("Sum = %d\n", sum);
    printf("Average = %.2f\n", average);

    return 0;
}

```

6. Exam Scoring Points & Memory Revision

★ Must-Write Points

1. Contiguous memory calculation: Address of arr[i] = Base Address + i * sizeof(type).
2. Array indexing starts at 0 and ends at N-1. Accessing arr[N] causes undefined behavior / memory corruption.
3. 2D Arrays: Stored in Row-Major order in memory (Row 0 elements followed by Row 1).
4. Passing arrays to functions decays them into pointers to the first element (int* arr).
5. Standard operations: Traversal O(N), Search O(N), Reversal O(N), Merge O(N+M).

⚡ Quick Recall

Base Address + (i * size) -> Contiguous Stack -> Row-Major 2D -> Pointer Decay -> Bounds Check

